

EXPERIMENT 3

DEBUGGING TOOLS IN C PROGRAMMING

Name	Ritesh Bansal
Student ID	590036924
Date	26 AUG 2026

Aim

To understand different types of errors in C and learn systematic debugging using compiler diagnostics, warning flags, printf-based tracing, and GDB.

Basic Concepts

Error: A mistake made by the programmer in source code. Example: Writing `a+b` when intention was to multiply `a*b`.

Bug: A flaw that causes incorrect, unexpected, or unintended program behavior. Example: The program outputs the sum of two numbers instead of their products.

Debugging: The systematic process of finding, analyzing, and removing bugs or defects in a program.

Classification of Errors

- Syntax errors - violations of C grammar, such as a missing semicolon.
- Compilation / type errors - incompatible types or missing declarations.
- Linker errors - object files are produced, but required definitions cannot be linked.
- Runtime errors - failures during execution, such as division by zero or invalid memory access.
- Logical errors - the program runs but produces an incorrect result.

GDB Setup and Common Commands

```
sudo apt update
sudo apt install gdb
cc -g -Wall -Wextra program.c -o program
gdb ./program
```

Useful GDB commands: break (b), run (r), next (n), step (s), print (p), display, continue (c), list (l), info locals, backtrace (bt), info breakpoints, delete, disable, enable, and quit (q).

Case 1: Syntax & Compilation Error

The compiler reports invalid C syntax and identifies the approximate location of the problem.

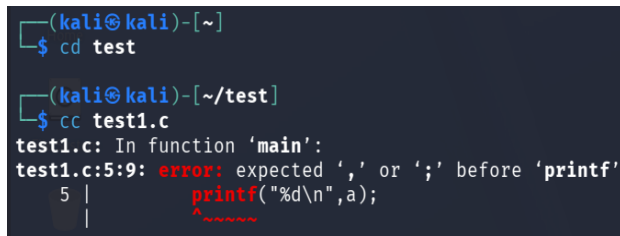
```
#include <stdio.h>
int main()
{
    int a = 10
    printf("%d\n", a);
    return 0;
}
```

Compilation / Debugging Steps

cc test1.c

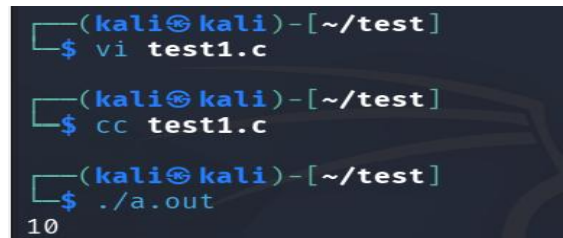
Observe the compiler diagnostic:

error: expected ',' or ';' before 'printf'

A terminal window on a Kali Linux system. The user navigates to a directory named 'test' and runs 'cc test1.c'. The compiler outputs an error: 'test1.c: In function 'main': test1.c:5:9: error: expected ',' or ';' before 'printf''. The error points to line 5, column 9, where the 'printf' statement is located. The code in the terminal matches the provided code block.

```
(kali㉿kali)-[~]
$ cd test

(kali㉿kali)-[~/test]
$ cc test1.c
test1.c: In function 'main':
test1.c:5:9: error: expected ',' or ';' before 'printf'
  5 |         printf("%d\n",a);
    |         ^~~~~~
```

A terminal window on a Kali Linux system. The user runs 'vi test1.c' to edit the file, then 'cc test1.c' to compile it. Finally, they run './a.out' to execute the program, which outputs the number '10'.

```
(kali㉿kali)-[~/test]
$ vi test1.c

(kali㉿kali)-[~/test]
$ cc test1.c

(kali㉿kali)-[~/test]
$ ./a.out
10
```

Explanation

The declaration `int a = 10` is missing a semicolon. The compiler reports the filename, line/column and an error description. Add the missing semicolon and compile again.

Corrected Code

```
#include <stdio.h>
int main()
{
    int a = 10;
    printf("%d\n", a);
    return 0;
}
```

Expected Result After Correction

10

Case 2: Compiler Warnings (-Wall, -Wextra)

Warnings identify suspicious code that is legal C but may indicate a bug.

Code (hand typed in the report)

```
#include <stdio.h>
int main()
{
    int a;
    printf("Hello\n");

    return 0; }
```

Compilation / Debugging Steps

```
cc test2.c
./a.out
cc -Wall -Wextra test2.c
# warning: unused variable 'a' [-Wunused-variable]
```

```
(kali㉿kali)-[~/test]
$ cc test2.c

(kali㉿kali)-[~/test]
$ ./a.out
Hello

(kali㉿kali)-[~/test]
$ cc -Wall -Wextra test2.c
test2.c: In function 'main':
test2.c:4:13: warning: unused variable 'a' [-Wunused-variable]
     4 |         int a;
       |             ^
```

```
(kali㉿kali)-[~/test]
$ vi test2.c

(kali㉿kali)-[~/test]
$ cc -Wall -Wextra test2.c

(kali㉿kali)-[~/test]
$ ./a.out
Hello
```

Explanation

Normal compilation may not show minor warnings. -Wall enables a broad set of useful warnings and -Wextra enables additional warnings. Here, variable a is declared but never used.

Corrected Code

```
#include <stdio.h>
int main()
{
    printf("Hello\n");
    return 0;
}
```

Expected Result After Correction

```
Hello
```

Case 3: printf-Based Debugging (Logical Bug)

printf statements can trace program flow and variable values before using a debugger.

Code (hand typed in the report)

```
{
    int i, sum = 0;
    for(i = 0; i < 5; i++)
#include <stdio.h>
int main(void)
{
    printf("DEBUG: entering iteration i = %d\n", i);
    sum = sum + i;
    printf("DEBUG: sum updated to = %d\n", sum);
}
printf("Final Sum = %d\n", sum);
return 0;
}
```

Compilation / Debugging Steps

```
cc test4.c -o test4
./test4
# Observe i and sum on every iteration.
```

```

(kali㉿kali)-[~/test]
$ cc test4.c -o test4

(kali㉿kali)-[~/test]
$ ./test4
DEBUG: i = 0
DEBUG: sum = 0
DEBUG: i = 1
DEBUG: sum = 1
DEBUG: i = 2
DEBUG: sum = 3
DEBUG: i = 3
DEBUG: sum = 6
DEBUG: i = 4
DEBUG: sum = 10
Final Sum = 10

```

```

(kali㉿kali)-[~/test]
$ vi test4.c

(kali㉿kali)-[~/test]
$ cc test4.c -o test4

(kali㉿kali)-[~/test]
$ ./test4
DEBUG: i = 1
DEBUG: sum = 1
DEBUG: i = 2
DEBUG: sum = 3
DEBUG: i = 3
DEBUG: sum = 6
DEBUG: i = 4
DEBUG: sum = 10
DEBUG: i = 5
DEBUG: sum = 15
Final Sum = 15

```

Explanation

The trace shows that `i` takes the values 0, 1, 2, 3 and 4. This exposes the logical problem when the intended calculation is the sum from 1 to 5. Change the loop to start at 1 and include 5.

Corrected Code

```

#include <stdio.h>
int main(void)
{
    int i, sum = 0;
    for(i = 1; i <= 5; i++)
        sum = sum + i;
    printf("Final Sum = %d\n", sum);
    return 0;
}

```

Expected Result After Correction

Final Sum = 15

Case 4: Debugging a Logical Error using GDB

A breakpoint and variable displays reveal why the loop gives 10 instead of the expected 15.

Code (hand typed in the report)

```

#include <stdio.h>
int main()
{
    int i, sum = 0;
    for(i = 1; i < 5; i++)
    {
        sum = sum + i;
    }
    printf("Sum = %d\n", sum);
    return 0;
}

```

Compilation / Debugging Steps

```

cc -g test8.c -o test8
gdb ./test8
(gdb) list
(gdb) break 9

```

```

(gdb) run
(gdb) print i
(gdb) display i
(gdb) display sum
(gdb) continue

(gdb) continue

(gdb) continue

(gdb) continue
# Continue until the program exits.

```

```

(gdb) list
1      #include <stdio.h>
2
3      int main()
4      {
5          int i, sum = 0;
6          for(i = 1; i < 5; i++)
7          {
8              sum = sum + i;
9          }
10
(gdb) break 9
Breakpoint 1 at 0x1151: file test8.c, line 9.
(gdb) run
Starting program: /home/kali/test/test8
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at test8.c:9
9      sum = sum + i;
(gdb) print i
$1 = 1
(gdb) display i
1: i = 1
(gdb) display sum
2: sum = 0
(gdb) continue
Continuing.

Breakpoint 1, main () at test8.c:9
9      sum = sum + i;
1: i = 2
2: sum = 1
(gdb) continue
Continuing.

Breakpoint 1, main () at test8.c:9
9      sum = sum + i;
1: i = 3
2: sum = 3
(gdb) continue
Continuing.

Breakpoint 1, main () at test8.c:9
9      sum = sum + i;
1: i = 4
2: sum = 6
(gdb) continue
Continuing.

Breakpoint 1, main () at test8.c:9
9      sum = sum + i;
1: i = 5
2: sum = 10

```

```

(kali@kali)-[~/test]
$ vi test8.c

(kali@kali)-[~/test]
$ cc test8.c -o test8

(kali@kali)-[~/test]
$ ./test8
Sum = 15

```

Explanation

GDB shows that the loop body executes for $i = 1$ through 4. When i becomes 5, the condition $i < 5$ is false, so 5 is never added. Change $i < 5$ to $i \leq 5$.

Corrected Code

```

#include <stdio.h>
int main()
{
    int i, sum = 0;
    for(i = 1; i <= 5; i++)
        sum = sum + i;
    printf("Sum = %d\n", sum);
    return 0;
}

```

Expected Result After Correction

Sum = 15

Case 5: Debugging Functions (next vs. step)

This case demonstrates the difference between stepping over and stepping into a function.

Code (hand typed in the report)

```
#include <stdio.h>
int square(int x)
{
    int result1;
    result1 = x * x;
    return result1;
}
int main()
{
    int result, n = 5;
    result = square(n);
    printf("Square = %d\n", result);
    return 0;
}
```

Compilation / Debugging Steps

```
cc -g test5.c -o test5
gdb ./test5
(gdb) break main
(gdb) run
(gdb) next
(gdb) next
(gdb) print result
# Restart for step demonstration:
(gdb) run
(gdb) next
(gdb) step
(gdb) print x
(gdb) next
(gdb) print result1
```

```
(gdb) break main
Breakpoint 1 at 0x1156: file test5.c, line 10.
(gdb) run
Starting program: /home/kali/test/test5
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at test5.c:10
10         int result, n = 5;
(gdb) next
11         result = square(n);
(gdb) next
12         printf("Square = %d\n", result);
(gdb) print result
$1 = 25
```

```
(gdb) run
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/kali/test/test5
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at test5.c:10
10         int result, n = 5;
(gdb) next
11         result = square(n);
(gdb) step
square (x=5) at test5.c:5
5         result1 = x * x;
(gdb) print x
$2 = 5
(gdb) next
6         return result1;
(gdb) print result1
$3 = 25
```

Explanation

next executes a function call without entering its body. step enters the called function so its statements and local variables can be inspected line by line. For $n = 5$, square returns 25.

Expected Result After Correction

Square = 25

Case 6: Runtime Error using Backtrace (bt)

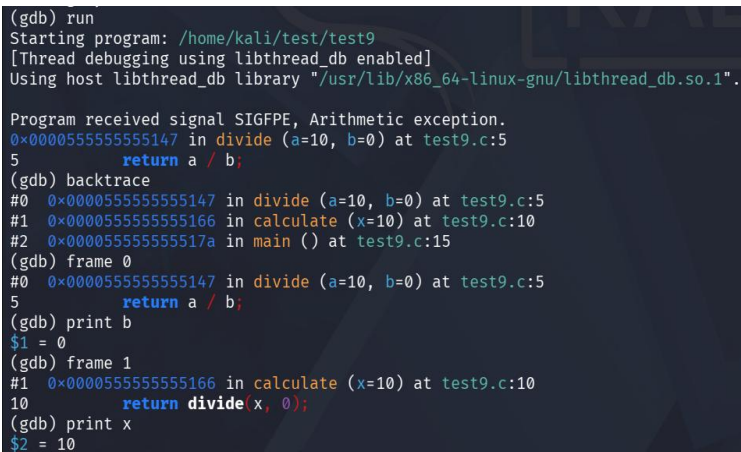
GDB backtrace identifies the chain of function calls that leads to a division-by-zero crash.

Code (hand typed in the report)

```
#include <stdio.h>
int divide(int a, int b)
{
    return a / b;
}
int calculate(int x)
{
    return divide(x, 0);
}
int main()
{
    int result = calculate(10);
    printf("Result: %d\n", result);
    return 0;
}
```

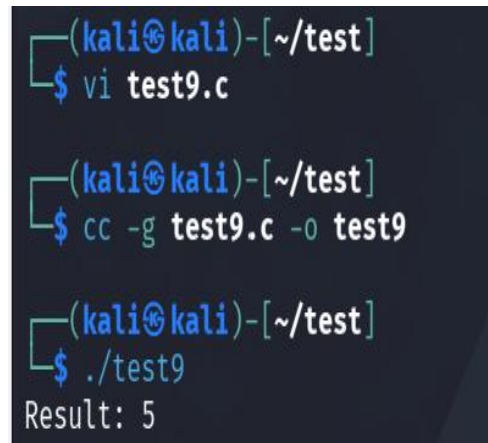
Compilation / Debugging Steps

```
cc -g test9.c -o test9
gdb ./test9
(gdb) run
# Program receives SIGFPE
(gdb) backtrace
(gdb) frame 0
(gdb) print b
(gdb) frame 1
(gdb) print x
```



```
(gdb) run
Starting program: /home/kali/test/test9
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGFPE, Arithmetic exception.
0x000055555555147 in divide (a=10, b=0) at test9.c:5
5      return a / b;
(gdb) backtrace
#0  0x000055555555147 in divide (a=10, b=0) at test9.c:5
#1  0x000055555555166 in calculate (x=10) at test9.c:10
#2  0x00005555555517a in main () at test9.c:15
(gdb) frame 0
#0  0x000055555555147 in divide (a=10, b=0) at test9.c:5
5      return a / b;
(gdb) print b
$1 = 0
(gdb) frame 1
#1  0x000055555555166 in calculate (x=10) at test9.c:10
10     return divide(x, 0);
(gdb) print x
$2 = 10
```



```
(kali@kali)-[~/test]
$ vi test9.c

(kali@kali)-[~/test]
$ cc -g test9.c -o test9

(kali@kali)-[~/test]
$ ./test9
Result: 5
```

Explanation

Frame #0 identifies `divide()` as the crash location and shows `b = 0`. Frame #1 shows `calculate()` passed zero as the divisor, while frame #2 is `main()`. The divisor must be non-zero.

Corrected Code

```
#include <stdio.h>
int divide(int a, int b)
{
    return a / b;
}
int calculate(int x)
{
    return divide(x, 1);
}
```

```

{
    return divide(x, 2);
}
int main()
{
    int result = calculate(10);
    printf("Result: %d\n", result);
    return 0;
}

```

Expected Result After Correction

Result: 5

Case 7: Array Out-of-Bounds Error using GDB

C does not automatically check array bounds, so index 5 is invalid for an array of five integers.

Code (hand typed in the report)

```

#include <stdio.h>
int main()
{
    int a[5];
    int i;
    for(i = 0; i <= 5; i++)
    {
        scanf("%d", &a[i]);
    }
    return 0;
}

```

Compilation / Debugging Steps

```

cc -g test10.c -o test10
gdb ./test10
(gdb) break 8
(gdb) run
(gdb) print &a[0]
(gdb) print &a[4]
(gdb) print &a[5]

```

```

(gdb) break 8
Breakpoint 1 at 0x1141: file test10.c, line 8.
(gdb) run
Starting program: /home/kali/test/test10
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1"

Breakpoint 1, main () at test10.c:8
8      for(i = 0; i < 5; i++)
(gdb) print &a[0]
$1 = (int *) 0x7fffffffd10
(gdb) print &a[4]
$2 = (int *) 0x7fffffffd20
(gdb) print &a[5]
$3 = (int *) 0x7fffffffd24

```

```

(kali@kali)-[~/test]
$ vi test10.c

(kali@kali)-[~/test]
$ cc -g test10.c -o test10

(kali@kali)-[~/test]
$ ./test10
56324
1 2 3 4 5

```


Explanation

An array declared as `a[5]` has valid indices 0 through 4. The condition `i <= 5` permits `i = 5` and accesses memory outside the array. Change the condition to `i < 5`.

Corrected Code

```
#include <stdio.h>
int main()
{
    int a[5];
    int i;
    for(i = 0; i < 5; i++)
        scanf("%d", &a[i]);
    return 0;
}
```

Case 8: Segmentation Fault (SIGSEGV) using GDB

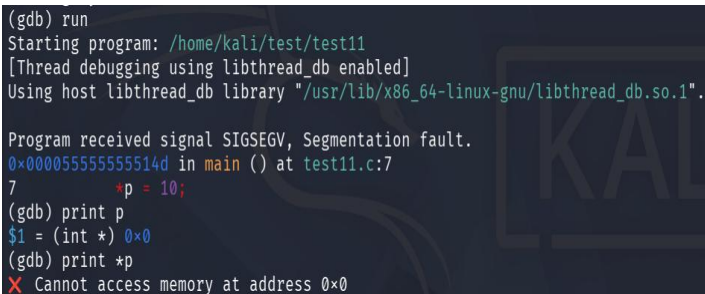
Dereferencing a NULL pointer attempts to access invalid memory and causes a segmentation fault.

Code (hand typed in the report)

```
#include <stdio.h>
int main()
{
    int *p = NULL;
    *p = 10;
    printf("%d\n", *p);
    return 0;
}
```

Compilation / Debugging Steps

```
cc -g test11.c -o test11
gdb ./test11
(gdb) run
# Program receives SIGSEGV
(gdb) print p
(gdb) print *p
```



```
(gdb) run
Starting program: /home/kali/test/test11
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
0x00005555555514d in main () at test11.c:7
7      *p = 10;
(gdb) print p
$p = (int *) 0x0
(gdb) print *p
X Cannot access memory at address 0x0
```



```
(kali@kali)-[~/test]
$ vi test11.c

(kali@kali)-[~/test]
$ cc -g test11.c -o test11

(kali@kali)-[~/test]
$ ./test11
10
```

Explanation

GDB shows `p = 0x0`. Dereferencing address `0x0` is invalid. Point `p` to valid memory before writing through it.

Corrected Code

```
#include <stdio.h>
int main()
{
    int x;
    int *p = &x;
    *p = 10;
    printf("%d\n", *p);
    return 0;
}
```

Expected Result After Correction

10

Systematic Debugging Strategy

1. Observe expected vs. actual output
2. Reproduce the issue consistently
3. Compile with flags (-Wall -Wextra -g)
4. Isolate suspicious code sections
5. Inspect state using printf or GDB breakpoints
6. Apply a single minimal fix
7. Recompile and re-test
8. Verify with edge cases and boundary inputs

Conclusion

The experiment demonstrates how compiler diagnostics, warning flags, printf tracing and GDB can be used to identify syntax, warning, logical, runtime, array-bound and memory-access problems. Correct debugging involves observing the failure, inspecting program state, making a minimal correction, recompiling and verifying the result.

Exercises & Homework Questions

1. Error Classification

Different types of errors in C are:

Syntax Error: We write C code against the rules of the language.

Example: forgetting a semicolon ;.

Compilation/Type Error: The compiler finds an incorrect use of data types or variables.

Example: trying to use an undeclared variable.

Linker Error: The program compiles, but the linker cannot find the definition of a function that is being called.

Runtime Error: The program compiles successfully but an error occurs while it is running. Example: division by zero.

Logical Error: The program runs without crashing but gives the wrong answer.

Example: using + when multiplication * was required.

2. GNU Concept

GNU is a project that provides free and open-source software and programming tools.

GDB (GNU Debugger) is one of the GNU tools. It helps programmers find errors in programs by allowing them to:

Stop the program at a particular line.

Execute the program step by step.

Check variable values.

Find where a program crashes.

3. Shell and Kernel

Kernel: The kernel is the core part of an operating system. It manages the CPU, memory, hardware, and other system resources.

Shell: A shell allows the user to give commands to the operating system. For example, when we type commands in a Linux terminal, the shell interprets them.

Three common Linux shells are:

Bash, Zsh, Fish

Easy way to remember:

User → Shell → Kernel → Hardware

4. Breakpoint, Watchpoint, Catchpoint and Tracepoint	
Breakpoint:	Stops the program at a particular line or function.
Watchpoint:	Stops the program when the value of a variable/expression changes.
Catchpoint:	Stops the program when a particular event occurs.
Tracepoint:	Collects information at a particular point without normally stopping the program.